What Limits Speculative Decoding at a Million Tokens? Greedy Loops, Rotary Stretching, and the Ring's Decode Step

Aman Kesarwani
Independent Researcher
amankesarwani@berkeley.edu

Abstract

Speculative decoding's short-context speedups do not survive million-token inference, and the reason is not the mechanism the technique's own literature would predict. We report three scientific findings and two systems measurements on 8×A100 80 GB SXM4 with FP4 weights and an NF4 KV cache. *First*, the acceptance such campaigns report is measured on degenerate text: under greedy decoding 9 of 10 rows in the released long-context first-window audit loop, and 1 of 5 rows in the 128 k greedy arm does, against 1 of 77 sampled rows at $T{=}0.7$. A greedy draft accepts a loop trivially, so a greedy acceptance number measures the text. *Second*, stretching the rotary basis of a model that ships the scaling dictionary costs **27 to 31%** of speculative acceptance over five books while a paired perplexity check on one passage, whose native perplexity is 1.013, moves by $+0.05\%$ and $+1.56\%$. Acceptance and perplexity dissociate, and on this model the perplexity check did not reveal the acceptance loss. *Third*, a natively 1M-trained target stays coherent at 1 M, within 9.5% of its own 128 k perplexity on the exactly aligned book. *Fourth*, our ring's decode bottleneck was not the ring but a host-side poll: 224 rounds of a `batch_isend_irecv` ring per token, each of which cost the same $\sim$10.3 ms under the blocking NCCL wait every run in this paper used, whatever the payload. Removing the rotation with decode-side context parallelism, which is prior work, is worth 2.80 to $4.38\times$ decode-only with speculation and 3.30 to $5.32\times$ without. *Fifth*, speculation still pays 1.54 to $2.29\times$ on top, and the matched 1 M acceptance is unchanged (-0.1%). Against vLLM at bf16 with greedy decoding, which is not a precision-matched comparison, vLLM's entire run takes 2.04 to $2.57\times$ our prefill while its decode stays 130 to $162\times$ faster per token than ours.

1 Introduction

Serving a million-token context on eight commodity GPUs forces two problems at once. The KV cache does not fit, which ring attention [Liu et al., 2023] solves by sharding the sequence dimension and rotating key/value blocks around the ranks. And every generated token still costs a full forward pass, which speculative decoding [Leviathan et al., 2023, Chen et al., 2023] attacks by having a small draft propose several tokens and verifying them with one target forward. We built **RASD**, a ring-attention engine with an NF4 KV cache, an attention-sink prefix, rotary scaling where a target needs it, and speculative verification, and asked whether speculation still pays at 1 M. The measurements kept saying that something else was wrong, so the findings below lead with the science and close on our own engine.

40th Conference on Neural Information Processing Systems (NeurIPS 2026). Workshop: Workshop on Long Context Foundation Models (LCFM) @ NeurIPS 2026.

F1. Acceptance and perplexity dissociate under rotary stretching. On `meta-llama/Llama-3.1-8B`, a model that ships the `rope_scaling` dictionary the intervention edits, stretching the rotary basis at 128 k costs **27 to 31 %** of speculative acceptance over five books, from 0.7272 to 0.5336 (factor 16) and 0.5006 (factor 32), while a paired perplexity check on a single passage moves by $+0.05\%$ and $+1.56\%$. The arms differ in nothing but `rope_factor`. Position scaling is judged by perplexity; on this model that measure did not reveal the acceptance loss.

F2. Greedy acceptance numbers at long context are measured on degenerate text. Under greedy decoding, 9 of 10 rows in the released long-context first-window audit degenerate into repetition, and 1 of 5 rows in the 128 k greedy arm does, against 1 of 77 rows sampled at $T{=}0.7$; that one row is a rotary variant and §4.1 names it. Rows are classified by their recorded temperature, not by their stage name. A greedy acceptance number therefore measures the degeneracy and not the agreement between draft and target, which is why this campaign moved to sampling and why every row here carries a periodicity verdict.

F3. A natively long-context target is coherent at 1 M. On the one exactly aligned book, the target's own perplexity at 1 M is 1.805 against 1.649 at 128 k, a $+9.5\%$ move; on the two books that share 91 to 95% of their scored tokens it is within 1.1%. Long context is not intrinsically hostile to speculation, which is what made the two systems findings below worth chasing.

F4. Our ring's decode step was a host-side poll, not the ring. At 128 k each decode token needs 7 rotations over the 32 patched layers, so 224 rounds of a `batch_isend_irecv` ring (`src/models/ring_attention_kernel.py:402` issues a rotation per `step < world_size - 1`). Every run in this paper measured that ring with PyTorch's NCCL blocking wait (`TORCH_NCCL_BLOCKING_WAIT=1`), in which each `wait()` polls its collective with a fixed 10 ms host sleep. A bare-process benchmark of the primitive, with no model attached, costs the same $\sim$10.3 ms per round at 4 KB and at 266 MB under that flag, and 0.134 ms rising to 2.619 ms (101.7 GB/s) without it: the flat line was the poll, and the wire is bandwidth-bound above a few hundred kilobytes. Removing the rotation with decode-side context parallelism, which is prior work [Yang et al., 2024], is worth **2.80**$\times$ **/ 3.77**$\times$ **/ 4.38**$\times$ decode-only with speculation and **3.30**$\times$ **/ 4.26**$\times$ **/ 5.32**$\times$ without it at 128 k / 512 k / 1 M on matched cells, and the time it removes grows with context, 2.43 to 13.88 s per token, because the shard a round ships grows by $8\times$ over that range. The flag could not simply be dropped: without it the engine emits a different token stream in the one speculation cell measured in both modes and fails an assertion in three of five stages, so split-KV without the flag was not measured.

F5. Speculation still pays, and the 1 M acceptance is unchanged. With the rotation gone, speculation is worth $1.54\times$ / $2.29\times$ / $2.20\times$ decode-only over the same engine's target-only path at 128 k / 512 k / 1 M, measured on matched books only: one book at 128 k, three at 512 k, five at 1 M. Acceptance at 1 M under the fixed engine shows no detectable net change against the corrected old-engine baseline, -0.1% over 13 matched cells with a 95% interval of about $\pm 12\%$ relative (per-cell changes run from -31% to $+30\%$), and the fixed kernel's target-only stream is bit-identical to the ring's for twelve tokens before diverging where the baseline's own top-2 margin is 0.0000.

What this paper does not claim. The rotation-free decode kernel is prior work, cited for the technique and never claimed: Meta's context-parallel report [Yang et al., 2024] introduces exact ring variants for million-token inference, including a pass-Q form that still circulates data around the ring each step, and vLLM's Decode Context Parallelism [Lee et al., 2026] is the production implementation of the rotation-free merge used here. Our contribution is the diagnosis and the measurement. Every timing in this paper was taken under the blocking NCCL wait of §3, and the comparison against vLLM is **not precision-matched** (§4.6).

2 Background and Related Work

Long-context attention. Ring attention [Liu et al., 2023] shards the sequence across ranks and pipelines exact attention through a ring of exchanges, so per-rank memory is independent of sequence length. The implementation here uses FlashAttention-2 [Dao, 2024] as the local kernel.

Ring Attention's own evaluation is about training and long-context capability; what a decode step costs once every layer's key/value shard must visit every rank is what §4.4 measures.

Decode-side context parallelism (prior work). The decode-phase gap is closed in the literature, and we cite rather than claim it. Meta's context-parallel report [Yang et al., 2024] introduces exact ring variants for million-token inference (pass-KV and pass-Q); in pass-Q the query travels around the ring to meet the shard it needs, followed by an all-to-all of partial outputs, so data still circulates once per step. What we adopt, and what vLLM ships as Decode Context Parallelism [Lee et al., 2026], is the rotation-free merge: the query is replicated, each rank attends to the shard it holds, and the partials are combined by log-sum-exp. We call that *split-KV*; the ring's per-step cost is what §4.4 is about.

Speculative decoding. Leviathan et al. [2023] and Chen et al. [2023] established draft-and-verify: a small model proposes γ tokens, the target verifies them in one pass, and the longest accepted prefix is kept with a residual resample for the rejected position. Acceptance α is the fraction the target accepts, and $(\alpha^{\gamma+1} - 1)/(\alpha - 1)$ is the expected number of tokens the target verifies per forward pass, not a wall-clock speedup; Cai et al. [2024] removed the separate draft model with multi-head prediction. All of it is evaluated at short context, and the interaction with a distributed KV cache is the setting here.

KV compression and position scaling. NF4 quantization [Dettmers et al., 2023] shrinks the cache and an attention-sink prefix [Xiao et al., 2024] stabilises it. Rotary embeddings [Su et al., 2024] are stretched past a model's training window by YaRN [Peng et al., 2023], an intervention that is evaluated by perplexity among other measures, and usually validated by a perplexity curve alone in practice; §4.2 shows what that misses. The intervention here is not YaRN: it changes the shipped `llama3` scaling factor of a model that already has one.

3 System and Setup

RASD in brief. RASD runs on one node of 8×A100 80 GB SXM4, with the target and a 1B draft (`meta-llama/Llama-3.2-1B`) on every rank. Its KV cache is NF4 with chunked updates and a bf16 attention-sink prefix of the first 128 tokens; attention is ring attention with FlashAttention-2 in each local block; the weights are **FP4**, since NF4 is the KV codec. At 1 M this campaign's rows record **29.6 GiB allocated and 32.8 GiB reserved on the worst rank** (35.6 GiB reserved on the spec row), while the vanilla HuggingFace stack runs out of memory at 128 k single-rank. The long-context target needs no rotary scaling at all (`rope_type: none`); where an arm does need it, the engine applies a fixed factor 8 at 8 k and above, anchored on each context's own length, because the `transformers` version in use ignores `original_max_position_embeddings`.

Targets. The long-context campaign runs the Gradient-1048k build of `meta-llama`'s Llama-3-8B (`gradientai`, "Gradient-1048k"), trained to a 1 M window, so nothing is stretched; the rotary intervention of §4.2 runs `meta-llama/Llama-3.1-8B`, which ships the scaling dictionary under test. A ceiling test on `meta-llama/Llama-2-7b-hf` is a capability reference only and no ratio here uses it (Appendix E).

Protocol. Prompts are PG-19 narrative [Rae et al., 2020], five books, windowed so each rung's scored continuation ends at the same document position as the 1 M rung, with the alignment verified from `continuation_sha256` and the recorded offsets rather than from a pool name. Decoding is T=0.7 sampling, 128 new tokens, γ=4 speculative steps, draft window cap 4096, seed 42 unless stated, and every row carries a repetition verdict from the detector of §4.1. Decode-only means $(t_{\text{row}} - t_{\text{ttft}})/(\text{tokens} - 1)$. Every fixed-engine row records the engine that produced it (`ring_decode_mode`, patched-layer count, git commit), so a row that cannot prove its engine is void.

Every timing here was taken under NCCL blocking wait. The pod env sets `TORCH_NCCL_BLOCKING_WAIT=1` (`scripts/mlsys_pod_env.sh:32`) and the campaign's remote scripts source it, so every `wait()` on a collective polls with a fixed 10 ms host sleep and a ring whose every round ends in a wait pays one poll period per round, which §4.4 measures.

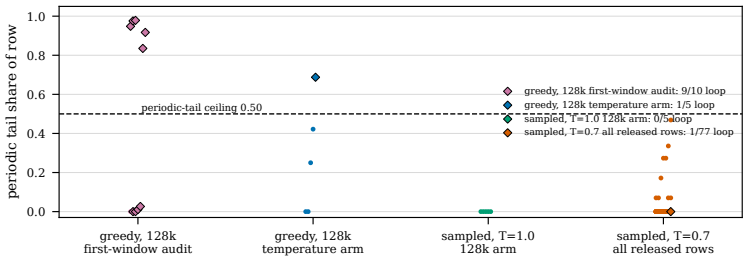


Figure 1: Periodic tail share per row for the rows that ship with a per-row verdict, against the 0.50 ceiling. Rows are grouped by their recorded temperature. Circles are rows the detector passes; diamonds are the rows it flags, which can sit below the share ceiling and still trip the 16-gram rule. Left to right: 9 of 10 greedy rows from the first-window audit loop, 1 of 5 in the 128 k greedy arm, 0 of 5 at T=1.0, and 1 of 77 sampled at T=0.7. The long-context census that motivated the move to sampling (9 of 15) is in Appendix A; its per-row sidecars are not part of this release, so it is quoted there and not used here.

Removing the variable is not a drop-in, because it exposes an ordering hazard in the ring's buffer reuse (§C); the numbers here describe the engine as it ran.

Draft conditioning. The draft sees the leading BOS plus the most recent 4095 tokens (`src/models/rasd_inference.py:878`, applied and logged at every draft prefill in the rows reported here).

4 Results

4.1 What a greedy acceptance number is measuring

The campaign behind this paper began under greedy decoding, and its acceptance numbers were high in a way that deserved suspicion: a draft that proposes the token the target would emit anyway is accepted, and a target that has fallen into a loop is maximally predictable. We measured the degeneracy instead of assuming it. The detector is the project's absolute repetition rule: a verbatim repeat span of 128 tokens or more, or 60% of tokens inside a repeated 16-gram, or a periodic tail covering 50% of the row.

Figure 1 is the census over the rows that ship with a per-row verdict, grouped by the temperature each row *records* rather than by its stage name: the 128 k temperature campaign ships two stages, one at T=0 and one at T=1.0. Under greedy, **9 of 10** rows in the long-context first-window audit loop and **1 of 5** in the 128 k greedy arm does; at T=1.0 none of five does; at T=0.7 **1 of 77** does. That row is `ROPE_EXT_SAMP_F16_SPEC_pg19_train_2768_s42`, a rotary arm on book 2768 with 77% of its tokens inside a repeated 16-gram and a zero periodic tail, which is why the figure marks it with the detector's verdict instead of the share. Six greedy rows with eight generated tokens each are excluded: too short for a detector whose shortest repeat span is 128 tokens. The looping row is inside the factor-16 mean of §4.2, which we therefore report with and without it.

Three consequences are applied here rather than merely noted. The headline protocol is sampled at T=0.7, with the earlier greedy numbers reported separately. A looping row is named wherever one appears, never smoothed into a mean. And the temperature ablation points *up*: at 128 k, T=1.0 reaches acceptance 0.7369 against 0.4681 at the headline T=0.7, with 0 of 5 rows degenerate in both arms.

4.2 Rotary stretching: acceptance and perplexity dissociate

The natural assumption is that an intervention which costs little in perplexity will cost little in acceptance, because the draft and the target see the same positions. We tested it on `meta-llama/Llama-3.1-8B` at 128 k, with five books per arm and the arms differing in nothing but `rope_factor`.

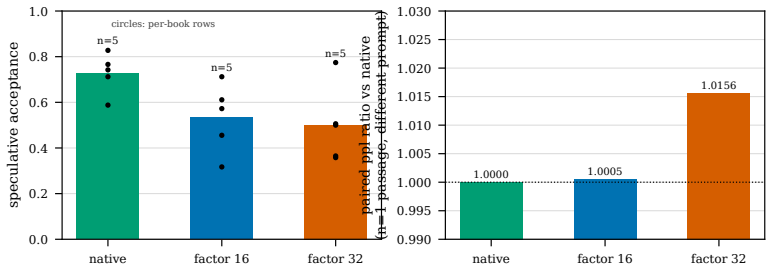


Figure 2: Left: speculative acceptance per arm, with the five books drawn as points so the direction can be checked per book rather than only in the mean. Right: the paired perplexity ratio against the native arm, from a single passage. Stretching the rotary basis costs 0.05% and 1.56% of perplexity on that passage and 27% and 31% of acceptance over five books.

Figure 2 is the measurement, and Table 4 the table. Acceptance falls from **0.7272** (native) to **0.5336** (factor 16) and **0.5006** (factor 32), a **27** to **31%** relative loss, while the paired perplexity ratio moves by $+0.05\%$ and $+1.56\%$. The direction holds on five books of five, so it is not a book effect, and because the arms differ in nothing else the drop is a **rotary effect across three arms**. One row needs naming: the factor-16 mean of 0.5336 contains `ROPE_EXT_SAMP_F16_SPEC_pg19_train_2768_s42`, the looping row of §4.1, whose acceptance of 0.7121 is the highest in its arm. Excluding it the factor-16 mean is 0.4890 over the remaining four books, a 32.8% loss, so the exclusion makes the effect larger, not smaller, and the claim uses the five-book mean and reports both. (6.6% apart) than either lands to native, which is what a saturating intervention looks like; at $n{=}3$ books their difference was 0.0012 and was read as exact saturation, and at $n{=}5$ it is 0.033, so that specific reading is withdrawn and the weaker claim stands.

A speculative decoder is only as good as the agreement between the draft's proposal distribution and the target's. Position scaling is normally evaluated by perplexity, and the one passage this campaign scored says it is nearly free here. The same intervention costs 27 to 31% of the acceptance over five books, and on this model the perplexity check did not reveal it, so acceptance under any change to the target's positional treatment has to be measured. The claim rests on the five-book acceptance contrast; the perplexity side is one passage and is reported as such rather than generalised. Every arm here ran on the old engine, so its absolute levels sit on the pre-fix decode path, and both arms paid the same blocking-wait cost.

4.3 Coherence at 1 M on a natively long-context target

The perplexity comparison above cannot reach 1 M, so we check the long-context property directly on a target trained to 1 M (`gradientai/Llama-3-8B-Instruct-Gradient-1048k`), where nothing is being stretched and the question is only whether the model is still a model. The rungs are matched by `continuation_sha256` and the recorded window offsets, never by a pool name.

On the **one exactly aligned book** the target's perplexity is **1.805 at 1 M** against **1.649 at 128 k**, a $+9.5\%$ move. That book is `pg19_train_0`, the King James Bible in the PG-19 pool, and its perplexity of 1.65 means the text is close to deterministic for this model, so it is the strongest and also the most memorised of the three. On the two books that share 91 to 95% of their scored tokens the difference is within **1.1%** at perplexities of 10.6 and 12.8. A natively long-context target therefore stays coherent across an eightfold context increase, which is the property the campaign depends on, though the strongest form of that claim rests on one near-deterministic book. The measurement is prefill-only, so the decode fix cannot move it.

4.4 The ring's key/value rotation was the decode step

At 128 k, each decode token requires 7 rotations over the 32 patched layers (`src/models/ring_attention_kernel.py:402` loops over the ranks and line 426 issues a rotation only while `step < world_size - 1`), so **224 rounds** of a `batch_isend_irecv` ring per token, measured under the NCCL blocking wait of §3, in which PyTorch's `WorkNCCL::wait` polls with a fixed 10 ms host sleep (`kSynchronizeBusyWaitMillis`, `ProcessGroupNCCL.cpp:426` in the pinned torch 2.5.1).

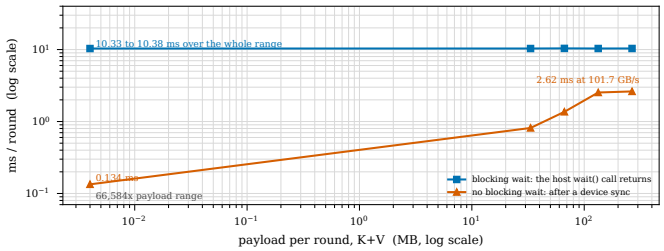


Figure 3: A bare-process benchmark of the ring primitive the engine uses (`batch_isend_irecv`, 8 ranks, 256 rounds per payload, no model), in both NCCL wait modes, same session and same box. Squares: the campaign's configuration, where the host `wait()` on each round returns after a fixed 10 ms poll, so the round costs the same at 4 KB and at 266 MB. Triangles: the same rounds with the wait flag removed and the timing taken after a device sync, where the round costs what the transfer costs and reaches 101.7 GB/s at 266 MB. NCCL selected `P2P/CUMEM/read` on a full NV12 mesh.

Table 1: Decode time per token, old ring engine against split-KV, on *matched* (seed, book) cells: one value per cell on both sides, plumbing repeats dropped, and the two seed stages hit by the configuration bug of §4.5 excluded. Speedup is the ratio of decode-only times. **Both columns were measured under NCCL blocking wait** (§3).

Context	Arm	Decode s/token		Speedup	n
		old ring	split-KV		(cells)
128k	spec	1.4779	**0.5275**	2.80×	3
128k	target-only	3.4925	**1.0596**	3.30×	1
512k	spec	3.5344	**0.9384**	3.77×	3
512k	target-only	9.9900	**2.3460**	4.26×	1
1M	spec	6.0506	**1.3830**	4.38×	13
1M	target-only	17.0902	**3.2107**	5.32×	2

Figure 3 measures that primitive with no model attached, over five payloads from 4 KB to 266 MB, in both wait modes in one session on one box.

Under the blocking wait every payload costs the same $\sim$10.3 ms: 10.3347 ms for 4 KB against 10.3520 ms for 266 MB, an **0.4% spread** across a 66,584× payload range. That flat line is the poll, not the wire. Without the flag the same primitive takes **0.134 ms** at 4 KB and rises with payload to **2.619 ms** at 266 MB, which is **101.7 GB/s**, and the two modes' post-sync times agree to within 3% at every payload, so the wire never changed.

Removing the rotation is prior work (§2), adopted here as **split-KV**: each rank computes attention for the replicated query against only its own prefill shard and the partials are merged by one MAX and one SUM all-reduce. Our contribution is the diagnosis and the measurement, not the technique. Split-KV's own times are blocking-wait times too, since its per-layer merges are synchronous collectives and pay their own share of the same poll.

Table 1 is the headline, with the same cells plotted in Appendix C. Decode-only, the fix is worth **2.80× / 3.77× / 4.38×** with speculation and **3.30× / 4.26× / 5.32×** without it at 128 k / 512 k / 1 M, and the mechanism is in what it removes rather than in the ratio. Dividing the removed time by the 224 rounds: at 128 k it is 2.43 s per token, or 10.9 ms per round, which is the poll period itself, and the microbench predicted $224 \times (10.3347 - 2.6191)$ ms $= 1.72$ s against a measured 1.75 s, within 1.4%. At 512 k and 1 M the removed time is far larger, 7.64 and 13.88 s per token, because the key/value shard a round ships grows by 4× and 8×: the transfer no longer fits inside one poll period, and under blocking wait the host cannot enqueue round $i+1$ until round i completes, so the next transfer cannot overlap the current compute. The rotation's cost scales with the data it ships, and the flat $\sim$10 ms per round is a 128 k-payload measurement.

Two caveats we state rather than bury. Every number here is a blocking-wait number, so it describes the engine as it ran rather than the hardware: the same ring decode step at 128 k is 1.93 s per token with the flag removed against 3.79 s with it, and split-KV was not measured in that mode

(Appendix C). cell at 512 k and two at 1 M, because that is how many books the old engine measured there, while the fixed engine's own target-only arms were widened to three and five books by a later top-up run.

4.5 Speculation still pays, and the baseline is corrected

With the rotation gone, the question the paper started from can be asked on an engine whose decode step is no longer dominated by a collective. The answer is yes, on **matched books only**, which matters at 128 k where the target-only arm is a single book: **1.54× / 2.29× / 2.20×** decode-only at 128 k / 512 k / 1 M over the same engine's own target-only path, over 1, 3 and 5 matched books. The 128 k figure is the one that changed: the older $2.01\times$ divided a three-book spec mean by a one-book target mean, and on the one book both sides measured it is $1.54\times$. The per-book values are in Appendix B. End-to-end, where prefill is included, the same comparison reads $1.50\times$ / $1.76\times$ / $1.41\times$, because at 1 M a row is 520 to 740 s depending on the arm, of which about 340 s is prefill: with the decode step fixed, prefill is the long pole and any statement about 1 M throughput has to say which of the two it means.

Two of those denominators used to be one and two books. We topped them up after the confirmatory run, because a ratio whose denominator is one row is a ratio of one row to another. The 512 k target-only mean is 2.148 s/token over three books against the 2.346 single-book value it replaces, and the 1 M mean is 3.155 over five against 3.211 over two. Both moves are downward, so the quoted ratios are lower than the thin ones: the thin denominator was flattering, by 8% at 512 k. The book-to-book spread inside the fixed engine is $1.27\times$ at 512 k and $1.30\times$ at 1 M, wider than the $1.02\times$ spread across seeds, so books and seeds are not interchangeable units.

The corrected old-engine baseline. Two stages of the seed ablation, the ones meant to run seeds 123 and 456 at 1 M, ran seed 42 instead, because the config builder takes the seed list from the defaults and then overwrites whatever a level declared: `run_experiment.py:693` pops `seeds` from `defaults`, line 713 merges the level over those defaults, and line 714 then sets `run["seed"]` from the default list, so the run ids those stages produced end in `_s42` (line 725) while their stage names claimed 123 and 456. Rebuilt from rows that were never affected, on 13 matched (seed, book) cells, the 1 M speedup is **4.38**× rather than $4.03\times$ and the 1 M acceptance difference is -0.1% rather than -10.4%: the fixed engine's acceptance is *unchanged*, and the previously reported drop was the bug. The old engine's own seed spread is 23.0% of the mean on the matched books, so per-seed movement of -15.7%, $+10.9\%$ and $+11.3\%$ sits inside the old engine's own seed-to-seed variation, in both directions.

The speedups are not bought with a different model. An exactly equivalent kernel can still move an acceptance number, because the accept/reject decision compares the target's distribution against the draft's proposal, so one flipped near-tie changes the emitted prefix and every later proposal with it. We checked the kernel directly: the target-only stream under split-KV is **bit-identical to the ring baseline for twelve consecutive tokens** and diverges at a step where the baseline's own top-2 logit margin is **0.0000**, against a row median of 2.9375, with prefill perplexity identical to six decimals.

4.6 Comparison against a production engine

We benchmark vLLM 0.6.3 at tensor-parallel 8 on the same prompts, fed the same token ids from our own sidecars, so the units match. The comparison is **not precision-matched**: vLLM runs **bf16 weights with a bf16 KV cache and greedy decoding**, while RASD runs **FP4 weights with an NF4 KV cache** at $T{=}0.7$ with speculation, and RASD's times are the split-KV engine measured **under NCLL blocking wait**. vLLM ran at its default chunked-prefill and batch-token settings, untuned. Every ratio below carries all three caveats.

Table 5 in Appendix B carries the numbers; two readings point in opposite directions. **vLLM's entire run takes 2.04× to 2.57× our prefill**, meaning its prefill plus its 128 decode steps against our first-token time; prefill against prefill the ratio is $2.52\times$ at 512 k and $2.56\times$ at 1 M. We do not quote a 128 k prefill ratio, because our own 128 k first-token time varied by about 40% between sessions with an identical configuration (8.7 to 12.4 s), as large as the ratio itself; that prefill advantage is a property of the ring prefill and is untouched by the decode fix. And **vLLM's decode is 130 to 162× faster per token than ours even after the fix**, against 429 to $731\times$ before it: the fix removed 70 to

81% of the decode deficit and left our decode far behind a production engine, and speculation does not close the gap (our spec decode is $65\times$, $65\times$ and $59\times$ vLLM's).

5 Limitations

- **Sample size.** The matched comparison is 3 spec and 1 target-only cell at 128 k, 3 and 1 at 512 k, 13 and 2 at 1 M, and the 128 k spec-over-target ratio is quoted on the one book both sides measured. The five units in a five-book arm are books, not seeds, and the book spread is wider than the seed spread.
- **Every timing was taken under NCCL blocking wait** (§3), which inflates the decode times of both engines on this path and of split-KV's own synchronous merges; split-KV was not measured without it. The flag is also not a free win, because removing it makes the engine emit a different token stream in one speculation cell and fail a device-side assertion in three of five diagnostic stages. This work reports that hazard and does not fix it (Appendix C).
- **One node, one fabric, one precision.** Everything is 8×A100 80 GB SXM4 over NVLink, there is no bf16 row, and the per-round quantity §4.4 measures is a host-side poll at this configuration, so it is not extrapolated to a multi-node fabric.
- **Acceptance under split-KV is explained, not tested.** The 1 M acceptance is unchanged over 13 matched cells while per seed it moves both ways inside a 23.0% old-engine spread; the mechanism we offer, one flipped near-tie changing the emitted prefix, was not tested directly.
- **Prefill is now the 1 M pole** and the perplexity side of F1 is one passage. A 1 M row is 520 to 740 s depending on the arm, about 340 s of it prefill, so the end-to-end 1 M speedup is $1.41\times$ against $2.20\times$ decode-only; and the perplexity ratio was scored on a different prompt from the acceptance rows at a native perplexity of 1.013, so the F1 claim rests on the acceptance contrast over five books.
- **An unattributed elementwise term.** In the profiled 128 k decode window, 19.3% of device time was elementwise kernels never attributed to an operation; with the rotation gone that is a larger share of what remains.
- **Two withdrawn claims are recorded rather than deleted** (Appendix A): the "communication is at most 1.2% of wall time" claim, whose bucket recorded 0.31 to $0.37\times$ of the physical floor, and the $4.4\times$ "native context" speedup from one pair in one session.

6 Conclusion

We asked what limits speculative decoding at a million tokens, and the answer arrived in three parts, none of them about speculation itself. Under greedy decoding the acceptance numbers this kind of campaign reports are measured on degenerate text: 9 of 10 long-context rows in the released first-window audit loop, and 1 of 5 rows in the 128 k greedy arm, against 1 of 77 sampled rows. Once sampled at $T{=}0.7$, stretching a model's rotary basis costs 27 to 31% of the acceptance over five books for 0.05 to 1.56% of perplexity on one passage, so the perplexity check did not reveal the loss. A natively 1M-trained target stays coherent at 1 M, within 9.5% of its own 128 k perplexity on the exactly aligned book.

What was hostile was our engine, and the first thing we learned about it is that our diagnosis was the measurement condition. Rotating each layer's key/value shard around eight ranks costs 224 rounds per decode token, and each round ended in a host-side wait that, under the NCCL blocking wait every run here used, polls with a fixed 10 ms sleep: the same $\sim$10.3 ms per round at 4 KB and at 266 MB, and 0.134 ms rising to 2.619 ms once the flag is removed. Adopting decode-side context parallelism, which is prior work, is worth 2.80 to $4.38\times$ decode-only with speculation and 3.30 to $5.32\times$ without it, and the time it removes is the poll period at 128 k and the serialised transfer at 512 k and 1 M, growing from 2.4 to 13.9 s per token because the shard a round ships grows with the context. Speculation still pays 1.54 to $2.29\times$ on top on matched books, the matched 1 M acceptance is unchanged at -0.1%, and against vLLM at bf16 and greedy decoding, not a precision-matched comparison, vLLM's entire run takes 2.04 to $2.57\times$ our prefill while its decode stays 130 to $162\times$ faster per token: the rotation was the decode pathology, and the flag that made it look flat is a configuration detail we can name.

References

Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D Lee, Deming Chen, and Tri Dao. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv:2401.10774*, 2024.

Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.

Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations (ICLR)*, 2024.

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. *arXiv preprint arXiv:2305.14314*, 2023.

Seonghee Lee, Sungsoo Ha, Omri Almog, and Lucas Wilkinson. Efficient decode context parallelism with vLLM for long context workloads. vLLM Blog, August 2026. URL `https://vllm.ai/blog/2026-08-07-decode-context-parallelism`. Accessed 2026-10-13.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning (ICML)*, 2023.

Hao Liu, Matei Zaharia, and Pieter Abbeel. Ring attention with blockwise transformers for near-infinite context. *arXiv preprint arXiv:2310.01889*, 2023.

Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. YaRN: Efficient context window extension of large language models. *arXiv preprint arXiv:2309.00071*, 2023.

Jack W Rae, Anna Potapenko, Siddhant M Jayakumar, Chloe Hillier, and Timothy P Lillicrap. Compressive transformers for long-range sequence modelling. In *International Conference on Learning Representations (ICLR)*, 2020.

Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations (ICLR)*, 2024.

Amy Yang, Jingyi Yang, Aya Ibrahim, Xinfeng Xie, Bangsheng Tang, Grigory Sizov, Jeremy Reizenstein, Jongsoo Park, and Jianyu Huang. Context parallelism for scalable million-token inference. *arXiv preprint arXiv:2411.01783*, 2024.

A Changes from the reviewed version

Table 2 lists every claim in the reviewed submission, with its status here. Corrections cite the audit item that found them (`results/mlsys/LCFM_RECONCILE.md`, AUDIT E1 to E23).

B The corrected baseline and the matched-cell tables

The matched-cell table the main text quotes is Table 1 (§4.4); it is not duplicated here. The two tables below carry the corrected old-engine seed baseline and the top-up rows that widen the two thin denominators.

The configuration bug. `run_experiment.py:693` takes the seed list from `defaults`, line 713 merges a level over those defaults, and line 714 then sets `run["seed"]` from the defaults' list, so a level that declares its own `seeds` has it recorded and then overwritten. Exactly two stages did: `SA_SEED_123_1M_SPEC` and `SA_SEED_456_1M_SPEC`, both of which ran seed 42 while their *stage names* said 123 and 456; the run ids those stages produced end in `_s42` (line 725), so the ids did not give the error away. Every old-engine pool containing them inherited it. The corrected baseline (Table 3) is built from rows that were never affected, seed 42 from the sampling session and seeds 123 and 456 from a dedicated rerun; the exclusion list is printed by `scripts/mlsys_seed_baseline_audit.py` and reproduced in `results/mlsys/SEED_BASELINE_AUDIT.txt`.

Effect. The 512 k target-only mean moves from 2.3460 over one book to 2.1480 over three, and the 1 M mean from 3.2107 over two books to 3.1553 over five. The spec-over-target ratios in §4.5 are computed on these wider arms; the matched speedups in Table 1 are not affected, because the new books have no old-engine counterpart. Both movements are downward, so widening the denominator lowered the reported ratios rather than raising them.

C The decode diagnosis in detail

This appendix holds the evidence the main text summarises, including the blocking-wait diagnostic that changed the F4 mechanism.

The five payloads span a factor of 66,584 in bytes and 0.4% in time under blocking wait, against 0.134 ms rising to 2.619 ms (101.7 GB/s) without it. The campaign's flat line was therefore the poll, and the fix is to remove the rounds rather than to move more data per round.

Two findings from that session belong to the honest version of F4. The token streams are **bit-identical across wait modes** in every target-only cell measured in both (128 k `ring2048` and `ring32768`, and 1 M `ring2048`), and the same digest reproduces across two sessions and two boxes; the one **speculation** cell measured in both modes diverges at token index 8, on a greedy seeded row against the same prompt. Removing the flag also produced a `indexSelectLargeIndex` device-side assertion in three of five stages, and raised peak reserved memory at 1 M from 32.8 GiB to 57.0 GiB on the worst rank. The flag was therefore masking an ordering hazard in the ring's buffer reuse, and it is not a free performance switch; this work reports it and does not fix it.

Correctness. The target-only stream under split-KV is bit-identical to the ring baseline for twelve consecutive tokens and then diverges at a step where the baseline's own top-2 logit margin is **0.0000**, against a row median of 2.9375 and a minimum of 0.0000: an exact tie broken the other way. Greedy repeats the pattern, and the speculative row diverges at index 1 with margin 0.8750 against a row median of 0.9375, below the row's own median. Prefill perplexity is identical to six decimals under both configurations, and the perplexity gate is in any case blind to a decode-path change, because it scores a prefill forward with `use_cache=False`.

Peak memory, worst rank. The main text's memory figure is this campaign's, not the reviewed build's trace:

D Ablations kept from the reviewed version

The design-point sweeps are **old-engine** results on the **synthetic repeated paragraph** at 64 k on 8×A100 40 GB, three seeds on one prompt, so the bracketed figures are ranges over those three runs and not confidence intervals. They are kept because they document how the system's knobs were chosen, and because two of the reviewed version's readings of them were wrong: the chunk size is a transmission chunk and does not bound memory, and prefetch depth 2 executes the same code path as depth 1, so that axis had two arms and not three. The block-size effect is specific to the ring decode path and is inert under split-KV. These runs predate the draft-window fix of §3 and are affected by it (Appendix A, E6); no claim here rests on their acceptance.

E Reproduction and evidence tree

Where every number lives. Nothing in this paper is transcribed by hand. The figures and tables are written by `scripts/lcfm_figures.py`, which reads the CSVs below and writes `results/lcfm/figs/` and `results/lcfm/tables/`; those tables are *included* by this document rather than copied into it.

- **Rotary intervention** `results/mlsys/sampling_session/session_a_ROPE_SAMP_*_SPEC.csv` and `results/mlsys/n5_extension/session_a_ROPE_EXT_*`, with the paired perplexity ratios in `session_a_rope_fix_gate.csv`.
- **Degeneracy** the per-stage `repetition_*.csv`, produced by `scripts/mlsys_repetition_report.py` from the token sidecars with the ceilings in `src/analysis/repetition.py`; where no such CSV exists the row is scored from the sidecar by the same module.
- **Coherence** `QUALITY_FIX_{128K,1M}_*` with the alignment table `results/mlsys/sampling_session/alignment.csv`.
- **The decode fix** the confirmatory run in `results/mlsys/rerun_r1/` and `rerun_r2/` (31 ok rows, each recording its engine and commit), the diagnostic arms in `results/mlsys/decode_df/arms/df_arms.csv`, and the top-up in `results/mlsys/rerun_topup/`.
- **The bare ring** `results/mlsys/decode_df/ring_bench/ringbench.csv`, from `scripts/mlsys_ring_microbench.py`, and the two-mode version of the same benchmark in `results/mlsys/bw_diag_p1/bench/` (blocking and no-flag, same session), with the engine arms in `results/mlsys/bw_diag_p1/arms/` and `results/mlsys/bw_diag_p2/`. The diagnostic is written up in `results/lcfm/BLOCKING_WAIT_DIAG.md` and the token streams it compares are extracted into `results/mlsys/bw_diag_token_streams.json`.
- **The perplexity check behind F1** `session_a_rope_fix_gate.csv`: three rows, one passage, prompt sha `574e34…`, native perplexity 1.013, which is a different prompt from the acceptance rows in `session_a_ROPE_SAMP_*_SPEC.csv`.
- **Book identity** `data/processed/pg19_1m/documents.json` records the title behind each `pg19_train_*` id; `train_0` is the King James Bible.
- **The corrected baseline** `scripts/mlsys_seed_baseline_audit.py`, which prints the exclusion list and the matched-cell table.
- **vLLM reference** `results/mlsys/vllm_run2/vllm_baseline.csv` (bf16, bf16 KV, greedy, unit-matched on every row). The earlier run at `results/mlsys/vllm_run/` is retained but its first-token and decode clocks are absolute monotonic readings and are unusable; only its end-to-end numbers agree, within 2.4% at 128 k and 0.1% elsewhere.

The ceiling test, in one place. Vanilla HuggingFace `generate()` with FlashAttention-2, bf16 weights and a bf16 KV cache, single rank, no quantization and no sequence parallelism, on `meta-llama/Llama-2-7b-hf`: 32 k completes at 30.1 GiB and 7.25 tok/s, and every context of 128 k and above runs out of memory during prefill. The model differs from the targets used for the throughput results (§3), so this is a capability reference for the standard stack and nothing more; no ratio in this paper uses it.

Building this paper. `cd manuscript/lcfm` and compile `main.tex` with `latexmk` or `tectonic`; the bibliography is `references.bib`. The long version of this work, including the withdrawn material and the full Ablations, lives in `manuscript/arxiv/`.

Table 2: Status of every claim in the reviewed version. KEPT means it stands as written; CORRECTED gives the old reading and the new one; REMOVED means the claim is withdrawn and not replaced. Audit items are E1 to E23 of `results/mlsys/LCFM_RECONCILE.md`.

Claim in the reviewed version	Status	Reason
Communication at most 1.2% of wall (E1)	REMOVED	The bucket recorded 0.31 to $0.37\times$ of the physical wire floor, so it never saw the transfers.
Idle at or above 50% is collective wait (E2)	REMOVED	Measured outside the region the collectives run in.
The tick-gate barrier (E3)	REMOVED	Not the code that produced the reported numbers.
The rotation is hidden and the draft pass is free (E4)	CORRECTED	The rotation is issued before attention, and the claimed draft overlap was never implemented.
Positioning table and novelty framing (E5)	CORRECTED	Split-KV is prior work, and the decode-overlap cell was not implemented.
The draft saw the last 4096 tokens (E6)	CORRECTED	Reviewed build only: the draft saw the prompt's *first* 4096 tokens, so its acceptance levels are lower bounds. Fixed in `d89cf41`; every row here uses the fix.
Anchor is the model's original window (E7)	CORRECTED	Each context was anchored on its own length.
Factor $\lceil N/4096 \rceil$ (E8, E9)	CORRECTED	A fixed factor 8 at 8 k and above, and none below.
Verify-loop defects (E10)	CORRECTED	Per AUDIT E10.
Three independent seed samples (E11)	CORRECTED	All three seeds ran one prompt.
95% intervals at $n{=}3$ (E12)	CORRECTED	Ranges are reported, with the n.
Nominal 1 M context (E13)	CORRECTED	1,047,551 prompt tokens here against 1,002,984 in the reviewed build.
Speedup regresses to $0.94\times$ and $0.97\times$ (E14)	REMOVED	Decode tokens per round are flat, and the same-code rerun does not reproduce it.
A control explains the bimodality (E15)	REMOVED	The control differs from the long-context cells in five ways at once.
$4.4\times$ at native context (E16)	REMOVED	One pair in one session, above what its own acceptance can deliver.
The speedup formula (E17)	CORRECTED	Applied to the wrong quantity; throughput is tokens over time.
Peak memory per rank (E18)	CORRECTED	The reviewed version's figure was the peak on rank 0 only, from the earlier Llama-2 build (41.2 GiB allocated at a 1,002,984-token prompt). This campaign's 1 M rows record 29.6 GiB allocated and 32.8 GiB reserved on the worst rank (§3).
"NF4 weights" (E19)	CORRECTED	The weights are FP4; NF4 is the KV codec.
NF4 KV description (E20)	CORRECTED	Per AUDIT E20.
Design-point sweeps (E21)	CORRECTED	Chunk size does not bound memory, depth 2 repeats depth 1, and the γ optimum is unsupported.
Prompts and citations (E22)	CORRECTED	The synthetic prompt is not the prior-work protocol, and one citation does not exist.
Qualitative comparison (E23)	KEPT	Stands, with the audit's caveat.
Greedy census, 9 of 15 rows looping	KEPT, MOVED HERE	Its per-row sidecars are **not released**, so the main text quotes released numbers only.
Profiler time-breakdown table and figure	CUT	The comm bucket in it is below the physical floor, so it bounds nothing.
Llama-2 quantization and rotary perplexity table	CUT	Labelled by intent rather than configuration, and one claim cited a number that does not exist.
RULER infrastructure appendix	CUT	Infrastructure without a run grid supports no claim.

Table 3: The corrected old-engine baseline at 1 M, over the 13 matched (seed, book) cells where both engines have a row (Table 1 lists the matched counts). The old engine's spread across its own seeds is 23.0% of the mean on these books.

seed	old acceptance	fixed-engine acceptance	change
42	0.5246	0.4424	-15.7%
123	0.4656	0.5164	$+10.9\%$
456	0.4466	0.4969	$+11.3\%$
pooled over 13 cells	0.4839	0.4834	-0.1%

Table 4: The rotary intervention at 128 k on `meta-llama/Llama-3.1-8B`: five books per arm, seed 42, FP4 weights with an NF4 KV cache Acceptance is the mean over five books; the per-arm table is Table 4 in Appendix B. The perplexity ratio is one 1024-token continuation of *one* passage, on a prompt none of the acceptance rows use, whose native perplexity is 1.013; it is a paired measurement against the native arm on that passage and nothing wider. The `tok/s` column is **old-engine** throughput, reported for completeness only: the finding is the acceptance contrast, and no speedup is claimed from these arms.

Arm	n	Acceptance	ppl ratio (vs native)	tok/s *[old engine]*
native	5	0.7272	1.0000	0.9109
factor 16	5	0.5336	1.0005	0.7699
factor 32	5	0.5006	1.0156	0.7473

Table 5: vLLM (bf16, bf16 KV, greedy, default kernel settings) against the split-KV ring engine (FP4 weights, NF4 KV, T=0.7 sampled, NCCL blocking wait) on the same prompts. Prefill is the first-token time; the two ratio columns are vLLM's prefill and vLLM's *entire run* (prefill plus 128 decode steps) each divided by RASD's prefill; decode is per token. **Not a precision-matched comparison**.

	Prefill (s)		vLLM run /		Decode (s/token)	
Context	vLLM	RASD	RASD prefill (same quantity)	RASD prefill (same quantity)	vLLM	RASD split-KV
128k	16.85	8.76	1.92×	2.04×	0.0081	1.06
512k	222.80	88.56	2.52×	2.54×	0.0145	2.35
1M	859.60	335.34	2.56×	2.57×	0.0234	3.21

Table 6: Speculation against the same engine's target-only path, on matched books only, all under blocking wait. Decode-only is the ratio of s/token; end-to-end is the ratio of mean tokens per second over the same matched books, so it includes prefill. The 128 k target-only arm measured one book, so only that book is matched there. This is the table the main text quotes.

Context	matched books	spec s/token	target-only s/token	speedup
128k	1	0.6861	1.0596	1.54×
512k	3	0.9384	2.1480	2.29×
1M	5	1.4370	3.1553	2.20×

Table 7: The target-only top-up, which widens the fixed engine's own target-only denominators (Table 6). The plumbing gate row is omitted. The matched old-against-new cells are unchanged by it.

stage	book	context	decode s/token	provenance
`TP_1M_TARGET`	`train_915`	1 M	2.826	`split_kv`, 32 layers
`TP_1M_TARGET`	`train_2204`	1 M	3.126	`split_kv`, 32 layers
`TP_1M_TARGET`	`train_2768`	1 M	3.402	`split_kv`, 32 layers
`TP_512K_TARGET`	`train_1`	512 k	1.847	`split_kv`, 32 layers
`TP_512K_TARGET`	`train_915`	512 k	2.251	`split_kv`, 32 layers

Table 8: Bare-process ring benchmark: the `batch_isend_irecv` ring primitive the engine uses, 8 ranks, 256 rounds per payload, no model attached, both NCCL wait modes in one session on one box. Reported for rank 0; the worst rank-to-rank spread over all eight ranks is 0.056 ms. The blocking column is the campaign's configuration: the timing is the host `wait()` call, which under `TORCH_NCCL_BLOCKING_WAIT=1` polls with a fixed 10 ms sleep. The last column is the same round with the flag removed, timed after a device sync so that a transfer that has been set up but not completed is still counted.

Payload	size	ms/round	
	(MB)	blocking wait	no blocking wait
`tiny_4kb`	0.004	10.3347	**0.1340**
`mb33`	33.292	10.3398	**0.8135**
`k8_66mb_expand_after`	66.585	10.3802	**1.3633**
`mb133`	133.169	10.3442	**2.5306**
`mb266`	266.338	10.3520	**2.6191**

Table 9: The blocking-wait and no-flag engine arms, 128 k and 1 M, 16-token target-only rows and 32-token spec rows, one session per mode on one box, `ring2048` unless noted. Every row records its wait mode in an `nccl_env` column and its arm in `env_arm`. The two flagged stages marked n/a died on a device-side assertion.

Context	Arm	Engine	mode	decode s/token	TTFT (s)
128 k	blocking	`ring2048`	target	3.7805	8.9
128 k	blocking	`ring2048`	spec	2.8811	7.5
128 k	blocking	`ring32768`	target	2.9553	7.1
128 k	blocking	`ring32768`	spec	2.2914	7.1
128 k	blocking	`splitkv`	target	1.0706	7.0
128 k	blocking	`splitkv`	spec	0.9026	7.2
128 k	no blocking wait	`ring2048`	target	2.0319	9.9
128 k	blocking spec	`ring2048`	spec	2.8482	11.3
128 k	no blocking wait spec	`ring2048`	spec	1.4052	11.0
1 M	blocking	`ring2048`	target	16.4348	338.5
1 M	blocking	`ring2048`	spec	7.5637	332.6
1 M	no blocking wait	`ring2048`	target	15.9613	334.8

Table 10: What the rotation costs per rung, target-only arms on matched cells, measured under blocking wait (Table 1). The last column divides the removed time by the 224 rounds per decode token. The 128 k figure is the poll period; the larger figures at 512 k and 1 M are the transfer that blocking wait serialises, which grows with the shard.

Context	Arm	old ring (s/token)	split-KV (s/token)	removed (s/token)	removed per round (ms, ÷224)
128k	target-only	3.4925	1.0596	2.43	10.9
512k	target-only	9.9900	2.3460	7.64	34.1
1M	target-only	17.0902	3.2107	13.88	62.0

Table 11: The engine arms from the original diagnosis: one model load, four configurations, 128 k target-only at 16 tokens each, all under blocking wait.

configuration	target-only s/token	vs `ring2048`	vs `ring32768`	token stream
`ring2048`	3.5637	1.000×		baseline
`ring32768`	2.8749	1.240×	1.000×	bit-identical
`ring32768x`	2.7985	1.273×	1.027×	bit-identical
`splitkv`	**1.0011**	**3.560**×	**2.872**×	12 tokens, then a tie

Table 12: 1 M rows, peak memory on the worst of the eight ranks, from the CSVs. `ring2048`, 16-token target-only rows and one 32-token spec row. The reserved figure is what the allocator held; allocated is what the tensors used.

1M rows, worst rank	allocated (GiB)	reserved (GiB)
blocking, ring2048 target	29.6	32.8
blocking, ring2048 spec	29.6	35.6
no blocking wait, ring2048 target	29.6	57.0

Table 13: M3 ablation results (64 k context, 8×A100-SXM4-40 GB). Mean and range over 3 seeds: **three seeds on one synthetic prompt**, not three independent samples, so the bracketed intervals are ranges rather than confidence intervals (AUDIT E11, E12); horizontal rules separate the five sweep axes. Deterministic early-EOS rows with tokens_generated < 20 are excluded; see §D in this paper for the exclusion rule.

Axis	Level	Throughput (tok/s) [range]	Accept. [range]	n
A1: Draft model	TinyLlama-1.1B	0.84 [0.82, 0.85]	0.235 [0.226, 0.252]	3
	Sheared-1.3B	0.87 [0.84, 0.90]	0.253 [0.231, 0.277]	3
A2: Spec steps (k)	k=2	0.81 [0.74, 0.85]	0.381 [0.303, 0.424]	3
	k=4	0.86 [0.81, 0.91]	0.253 [0.231, 0.277]	3
	k=6	0.84 [0.80, 0.88]	0.176 [0.149, 0.193]	3
	k=8	0.86 [0.80, 0.89]	0.147 [0.124, 0.160]	3
	k=12	0.85 [0.82, 0.88]	0.111 [0.105, 0.119]	3
A3: KV block size	256	0.62 [0.58, 0.65]	0.253 [0.231, 0.277]	3
	512	0.87 [0.81, 0.93]	0.253 [0.231, 0.277]	3
	1024	1.11 [1.06, 1.15]	0.253 [0.231, 0.277]	3
	2048	1.25 [1.20, 1.30]	0.253 [0.231, 0.277]	3
A4: Prefetch depth	sync	0.88 [0.84, 0.90]	0.253 [0.231, 0.277]	3
	async-1	0.87 [0.81, 0.92]	0.253 [0.231, 0.277]	3
	async-2	0.86 [0.84, 0.89]	0.253 [0.231, 0.277]	3
A5: Target model	Llama-2-7B	0.86 [0.83, 0.90]	0.253 [0.231, 0.277]	3